# Burman Capability Platform

Sean Burman

# Contents

**Burman Capability Platform** **3**
A thesis on Capability Architecture, the current B2B stack, and the road from PlumbingTrack to governed organizational software . . . 3
Table of contents . . . 3
1. Thesis . . . 4
2. Philosophy: what Capability Architecture claims . . . 5
2.1 The central question . . . 5
2.2 Capability versus automation . . . 5
2.3 Foundational principles (P-001–P-010) . . . 6
2.4 The vocabulary that matters . . . 6
2.5 Ethics, uncertainty, and human purpose . . . 7
3. The Burman Capability Platform today . . . 7
3.1 Name and position . . . 7
3.2 What ships now: PlumbingTrack . . . 9
3.3 Current technical stack . . . 9
3.4 Compile-time modularity seam . . . 9
3.5 Request and data flow . . . 10
3.6 Domain model as proto–work objects . . . 11
3.7 Scheduling as a deterministic capability . . . 11
3.8 Auth, roles, and authority (partial) . . . 11
4. Mapping theory to code . . . 13
4.1 Reference architecture layers vs. repository reality . . . 13
4.2 Development roadmap alignment (Stages 0–6) . . . 13
4.3 Gap analysis . . . 14
5. Architecture diagrams . . . 14
5.1 As-is: composed server . . . 14
5.2 Bounded contexts (target) . . . 16
5.3 Capability loop (target) . . . 17
5.4 Multi-tenant B2B topology . . . 17
6. Implementation routes . . . 19
6.1 Route A — Harden the seam, ship PlumbingTrack . . . 19
6.2 Route B — Observation layer first . . . 19
6.3 Route C — Extract and multiply modules . . . 19
6.4 Route comparison matrix . . . 22
7. Design considerations . . . 23
7.1 MongoDB today, plural models tomorrow . . . 23

7.2 Events: what exists vs. what the theory requires . . . . . 23
7.3 HTML, templ, and operational interfaces . . . . . 23
7.4 Agents: when, not if—and how bounded . . . . . 24
7.5 B2B packaging: tenants, entitlements, white-label . . . . . 24
8. Future prospects . . . . . 25
8.1 Near term (6–12 months) . . . . . 25
8.2 Medium term (12–24 months) . . . . . 25
8.3 Long term: capability systems, not app catalogs . . . . . 25
9. Research program and falsification . . . . . 26
10. Conclusion . . . . . 27
11. Appendix: key sources . . . . . 27

# Burman Capability Platform

## A thesis on Capability Architecture, the current B2B stack, and the road from PlumbingTrack to governed organizational software

**Working document · July 2026 · v0.1**

> *The conventional object of software design is an application that encodes a workflow. The proposed object is an organizational capability system that observes work, represents changing context, develops governed interventions, and measures whether the organization became more able to achieve its purposes.*
>
> — *The Capability Architecture, Volume I: Foundations*

---

## Table of contents


1. Thesis
2. Philosophy: what Capability Architecture claims
   - 2.1 The central question
   - 2.2 Capability versus automation
   - 2.3 Foundational principles (P-001–P-010)
   - 2.4 The vocabulary that matters
   - 2.5 Ethics, uncertainty, and human purpose
3. The Burman Capability Platform today
   - 3.1 Name and position
   - 3.2 What ships now: PlumbingTrack
   - 3.3 Current technical stack
   - 3.4 Compile-time modularity seam
   - 3.5 Request and data flow
   - 3.6 Domain model as proto–work objects
   - 3.7 Scheduling as a deterministic capability
   - 3.8 Auth, roles, and authority (partial)
4. Mapping theory to code
   - 4.1 Reference architecture layers vs. repository reality
   - 4.2 Development roadmap alignment (Stages 0–6)
   - 4.3 Gap analysis
5. Architecture diagrams

- 5.1 As-is: composed server
- 5.2 Bounded contexts (target)
- 5.3 Capability loop (target)
- 5.4 Multi-tenant B2B topology

6. Implementation routes
   - 6.1 Route A — Harden the seam, ship PlumbingTrack
   - 6.2 Route B — Observation layer first
   - 6.3 Route C — Extract and multiply modules
   - 6.4 Route comparison matrix
7. Design considerations
   - 7.1 MongoDB today, plural models tomorrow
   - 7.2 Events: what exists vs. what the theory requires
   - 7.3 HTML, templ, and operational interfaces
   - 7.4 Agents: when, not if—and how bounded
   - 7.5 B2B packaging: tenants, entitlements, white-label
8. Future prospects
   - 8.1 Near term (6–12 months)
   - 8.2 Medium term (12–24 months)
   - 8.3 Long term: capability systems, not app catalogs
9. Research program and falsification
10. Conclusion
11. Appendix: key sources


---

## 1. Thesis

**Burman Capability Platform** (BCP; codebase: `bman-platform`) is the first deliberate implementation surface for *The Capability Architecture*—a computational theory of organizations in which **capability**, not application or dashboard, is the durable unit of value.

Today BCP is a working B2B operations stack: a Go web platform with a compile-time module seam, MongoDB persistence, role-based access, internal calendars, and a production tenant implementation (**PlumbingTrack**) for multi-stage residential repiping. It is ordinary software that happens to be structurally pointed at something larger.

The thesis of this document is threefold:

1. **Theory precedes implementation, but implementation must eventually speak back.** P-001 is not academic decoration. The platform exists to test whether observation-driven, capability-centered design produces better trade-operations software than feature-first ERP thinking.

2. **BCP is correctly positioned at Stage 0–1 on the Capability Architecture roadmap**—a stable operational core with emerging observation primitives (calendars, stage history, checklists, exceptions in the field) but not yet a canonical event layer, capability registry, or bounded agent runtime.

3. **The extraction path already begun in `platform.Module` is the right long-term shape** for a B2B product line: one governed platform, many compile-time capability mod-

ules (PlumbingTrack first; electrical, HVAC, or general contractor scheduling later), shared identity and calendar infrastructure, domain logic that never leaks upward.

The product does not need to *look* like AI to be architecturally serious. It needs to **produce governed observations**, preserve **work-object identity**, encode **authority**, and eventually close the loop from friction → capability → measured outcome. Everything else—models, agents, code generation—is downstream.

---

## 2. Philosophy: what Capability Architecture claims

### 2.1 The central question

Business software matured without maturing its relationship to the organization. A workflow is specified; an application is built; people bend their work to the UI. When reality shifts—a regulation, a superintendent leaving, a building with odd stack geometry—the software does not *notice*. It merely becomes wrong, and a project is funded to fix it.

Capability Architecture reframes the design question:

```
Not:  "What app should we build?"
But:  "How should an intelligent organization be represented computationally
       so software can observe operation, understand structure, and develop
       governed capabilities in response?"
```

Organizations are not finished specifications. They are **distributed decision systems**: bounded rationality (Simon), dispersed knowledge (Hayek), routines as carriers of organizational memory (Nelson & Winter). Software that ignores informal coordination, exceptions, and local expertise encodes a fiction—and then punishes users when the fiction breaks.

### 2.2 Capability versus automation

| Concept | What it is | What it is not |
|---|---|---|
| **Automation** | Executes a known procedure | Understanding when to stop |
| **Feature** | Describes software behavior | Proof of organizational ability |
| **Workflow** | Permitted state transitions | Empirical process history |
| **Capability** | Persistent ability to achieve a class of outcomes with less uncertainty, coordination, or expertise | A prompt, a dashboard, or a chat box |

A capability **recognizes applicability**, **acquires context**, **handles exceptions**, **uses authority**, **measures outcomes**, and **maintains itself over time**. That is why Capability Architecture treats capability as the fundamental unit of adaptive B2B software—not the microservice, not the LLM wrapper.

*Figure 2.1 — Data is material. Capability is the product (P-002).*

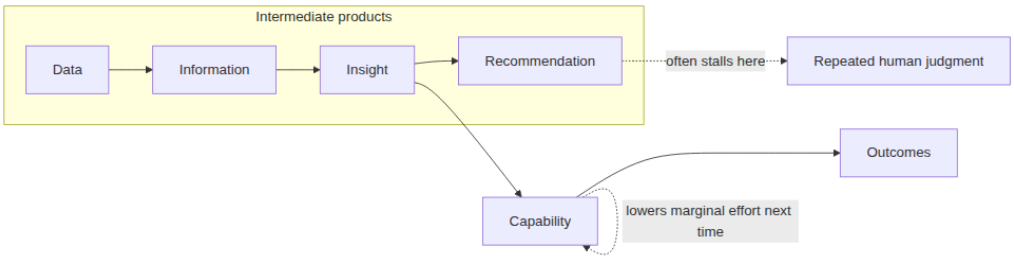



Figure 1: Architecture diagram 1

### 2.3 Foundational principles (P-001–P-010)

These are design commitments for BCP—not slogans. Each implies engineering consequences.

| ID | Principle | BCP implication |
|---|---|---|
| **P-001** | Theory precedes implementation | `docs/modularity.md` and this thesis define boundaries before convenience redefines them |
| **P-002** | Capability is the product | Success = reduced friction in real jobs, not feature count |
| **P-003** | Software evolves from observation | Stage units, exceptions, reschedules, and checklists are evidence—not bugs in requirements |
| **P-004** | Provenance and time preserved | Calendar migration (`EventID` → `InternalCalendarEventID`) is a proto–temporal concern; full event envelope still missing |
| **P-005** | Authority explicit and machine-enforceable | RBAC middleware exists; purpose-scoped retrieval and delegation do not |
| **P-006** | Bounded agent autonomy | No general assistant; future agents get mandates, tool contracts, audit streams |
| **P-007** | Outcomes and retirement paths | Scheduling rules versioned in code; no capability registry lifecycle yet |
| **P-008** | Plural organizational model | Mongo documents + calendar projections today; graph/event/document views later |
| **P-009** | Human purpose external to optimization | Metrics must not silently redefine job quality (e.g. "faster stage closure"  good work) |
| **P-010** | Uncertainty visible | UI should expose confidence and missing context before model-assisted features ship |

**P-001 in practice:** The manuscript explicitly separates theory from platform. `bman-platform` is *a possible implementation*, not the theory itself. That separation protects both: the theory can be revised without destabilizing production PlumbingTrack deployments, and PlumbingTrack can adopt Postgres, brokers, or model gateways without rewriting what a capability *means*.

### 2.4 The vocabulary that matters

Capability Architecture assigns precise meanings. BCP mapping uses these terms intentionally:

| Term | Manuscript definition | BCP instance today |
|---|---|---|
| **Organization** | Bounded network of people, roles, goals, routines | Plumbing contractor org; building admin; tenant |
| **Work object** | Entity work coordinates around | `Job`, `StageUnit`, `StageUnitRoom` |
| **Event** | Recorded occurrence asserting change | Calendar event create/move; API |

| Term | Manuscript definition | BCP instance today |
|---|---|---|
| **Agent** | Governed decision-making runtime | *Not yet*—humans + deterministic services only |

**Critical distinction (often lost in SaaS design):** an *event* is a record in the system; an *observation* is a claim about reality carried by that event. Submitting "rough-in complete" is an event; whether the physical work is complete is an observation with provenance and dispute potential. BCP currently conflates these in places—an honest gap, not a failure.

### 2.5 Ethics, uncertainty, and human purpose

Capability Architecture is not "AI for ops" cosplay. Three philosophical guardrails matter for B2B:

1. **Augmentation, not substitution** (Engelbart, Licklider). Remove coordinative load; preserve judgment. The workspace calendar assists scheduling; it does not replace superintendent discretion.
2. **Observation is power.** Embeds, access links, and room photos create legibility. They must serve declared operational purposes, respect role boundaries, and avoid collapsing human performance into a single proxy metric.
3. **The counterexample of the Omniscient Optimization Engine.** A platform that infers employee goals, ranks everyone continuously, and rewrites procedures for a corporate scalar metric is the *anti-pattern*—optimizing the representation instead of serving the organization.

BCP's embed tokens, frame-ancestors CSP, and role-separated API middleware are early structural ethics—not complete governance, but directionally correct.

---

## 3. The Burman Capability Platform today

### 3.1 Name and position

| Name | Role |
|---|---|
| **The Capability Architecture** | Versioned theory (manuscript v0.3) |
| **Burman Capability Platform (BCP)** | Product and platform line implementing the theory |
| `bman-platform` | Go module / repository for the shared server |
| **PlumbingTrack** | First `platform.Module` implementation—production B2B tenant |

Branding can evolve; the architectural commitment is **one platform, many capability modules, compile-time composition**.

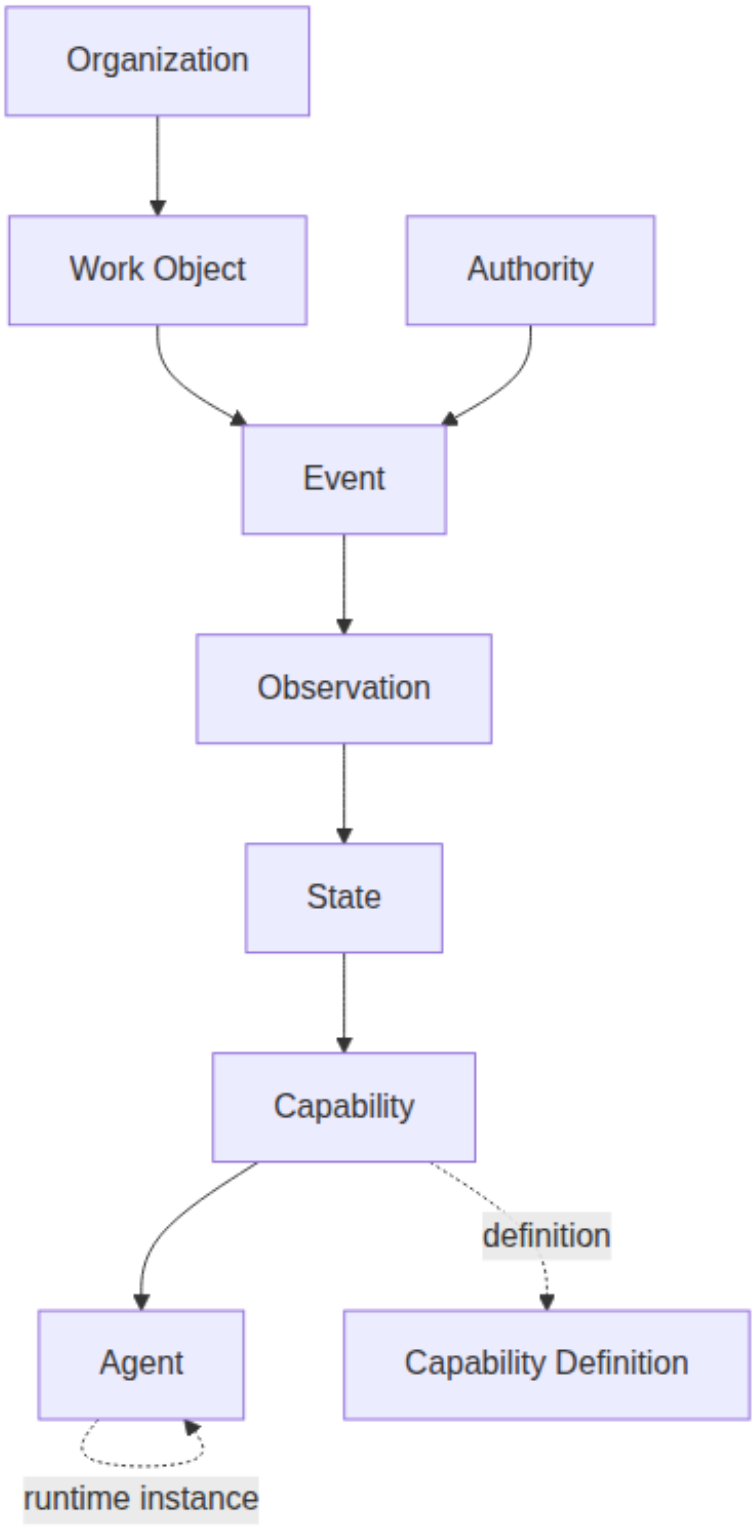



Figure 2: Architecture diagram 2

### 3.2 What ships now: PlumbingTrack

PlumbingTrack is scheduling and workflow software for **multi-stage plumbing repiping** in residential towers:

| Stage | Name | Typical work |
|---|---|---|
| 1 | Prep | Mapping, cutting, tracking, shop drawings |
| 2 | Rough-in | Rough-in, pressure test, inspection |
| 3 | Commissioning | Commissioning |
| 4 | Finishing | Drywall through painting |

**Jobs** link to internal MongoDB calendars. **Stage units** (suites) progress through the pipeline. **Rooms** carry measurements, pipe cuts, paint codes, progress, Cloudinary photos. Managers use a workspace calendar; field staff use access links; building admins and tenants get **embeddable read-only calendars**.

This is deliberately ordinary software. The differentiator is structural: shared identity, calendar primitives, modular domain boundary, and a scheduling engine that encodes real trade knowledge (block keys, prep→rough-in dependencies, working-day math, cascade rescheduling).

### 3.3 Current technical stack

| Layer | Technology | Notes |
|---|---|---|
| Language | Go 1.25 | Matches manuscript's conservative reference stack |
| HTTP | Echo v4 | Middleware, groups, static assets |
| Database | MongoDB | Document store; not yet event-sourced |
| Auth | bcrypt, HMAC sessions, encrypted email | Mongo-backed `app_sessions` |
| Media | Cloudinary | Room images |
| Frontend | TypeScript → JS, Tailwind CDN | Large `app.ts`; workspace calendar scripts |
| Templates | templ (CI-enforced) | Migration in progress from Go HTML strings |
| Deploy | Heroku-style Procfile | `web: scheduler` |

### 3.4 Compile-time modularity seam

The platform's central architectural bet is `platform.Module`:

```
type Module interface {
    Name() string
    Register(*echo.Echo, Dependencies) error
}
```

Boot sequence (`cmd/scheduler/main.go`):

```
Load .env → MongoDB → platform.New(...) → app.Use(plumbingtrack.New()) → Listen
```

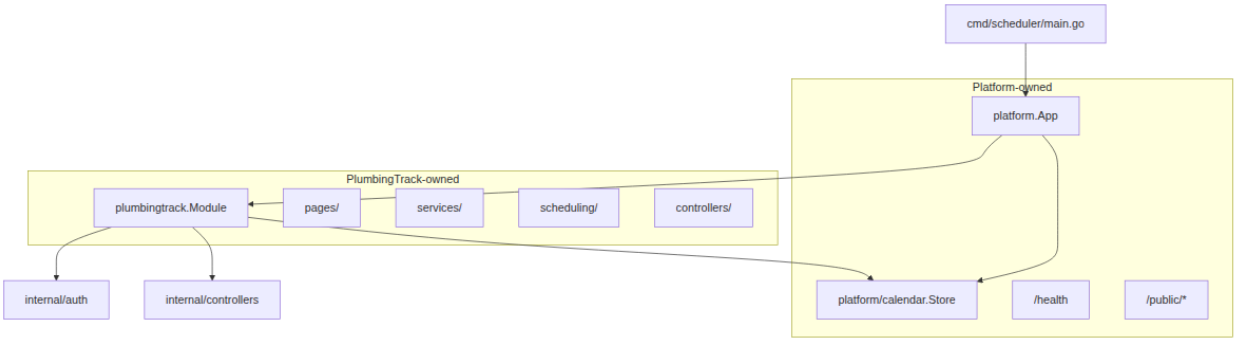



Figure 3: Architecture diagram 3

**Dependency rule (from `docs/modularity.md`):** suites, stage units, rooms, walls, paint, commissioning—**PlumbingTrack only**. Platform owns server startup, middleware, health, auth primitives, tenant context, calendar store interface, rendering helpers.

No runtime Go plugins. Future trade apps import `bman-platform` and register their module—same pattern, different binary.

### 3.5 Request and data flow

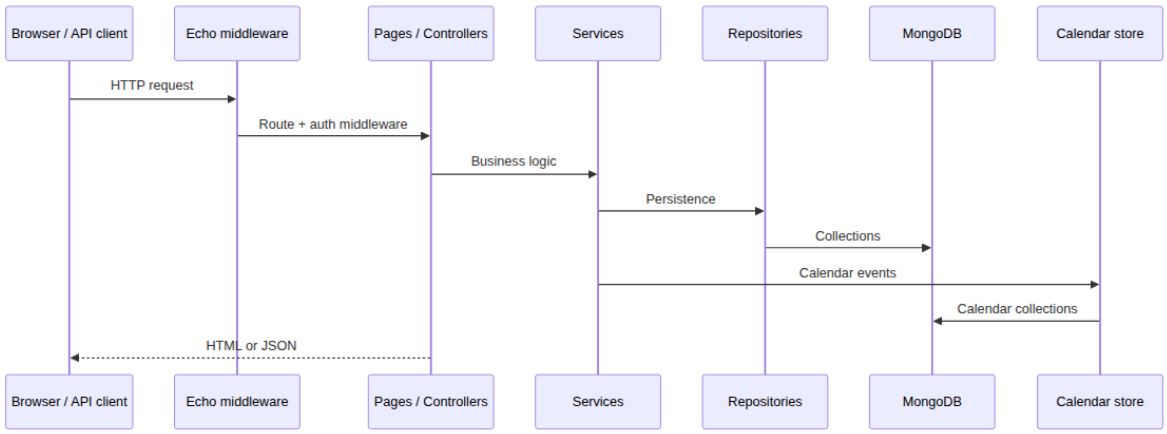



Figure 4: Architecture diagram 4

**Route surface (simplified):**

| Area | Prefix | Owner |
|---|---|---|
| Health | `GET /health` | `platform` |
| Auth | `/login`, `/admin/*`, `/access/:token` | `internal/controllers` |
| Calendars | `/calendars`, `/api/v1/calendars` | `internal/controllers` |
| Workflow UI | `/jobs`, `/stage-units`, workspace | `plumbingtrack/pages` |

| Area | Prefix | Owner |
|---|---|---|
| Embeds | `/embed/jobs/:jobId/*` | `plumbingtrack/pages` |
| REST | `/api/v1/jobs/.../stage-units` | `plumbingtrack/controllers` |

### 3.6 Domain model as proto–work objects

BCP does not yet use the manuscript's `WorkObject` type—but the mapping is clear:

| Manuscript concept | BCP entity | Identity |
|---|---|---|
| Work object (project) | `Job` | Mongo `ObjectID` |
| Work object (unit) | `StageUnit` | Per-job suite identity |
| Work object (room) | `StageUnitRoom` | Deduped room records |
| Workflow | 4-stage pipeline + calendar events | `scheduling/rules.go` |
| Exception | Manual reschedule, cascade, field workarounds | Partially captured |
| Outcome | Stage completion, inspection pass | Implicit in status fields |

**Friction signal (H-005):** repeated exception handling—duplicate rooms (hence `DedupeAllRooms` on boot), cascade toggles, manual checklist edits—are discovery material for future capabilities, not annoyances to hide.

### 3.7 Scheduling as a deterministic capability

`internal/plumbingtrack/scheduling/` is the clearest **proto-capability** in the codebase: versioned rules, deterministic output, explicit dependencies, persistence through `SchedulingService`.

```
BlockKey("301") → "01"     # stacks 101/201/301 share block
StagePrep → StagePlumbing  # rough-in lands week after prep batch
Working-day normalization  # weekends skipped
Cascade reschedule         # downstream stages move
```

This is **automation elevated toward capability**: it knows *when* the procedure applies (stage number, block topology), handles *dependencies* (prep events for rough-in), and persists results—but it lacks a formal capability definition, outcome metrics, or retirement/version contract per P-007.

### 3.8 Auth, roles, and authority (partial)

| Role | Access pattern |
|---|---|
| `owner` / `admin` | Full management, calendar edit, users |
| `plumber` / `finisher` | Access-link field workflow |
| `client` | Read-only calendar API (org-scoped) |
| `tenant` | Unit-specific tenant views |

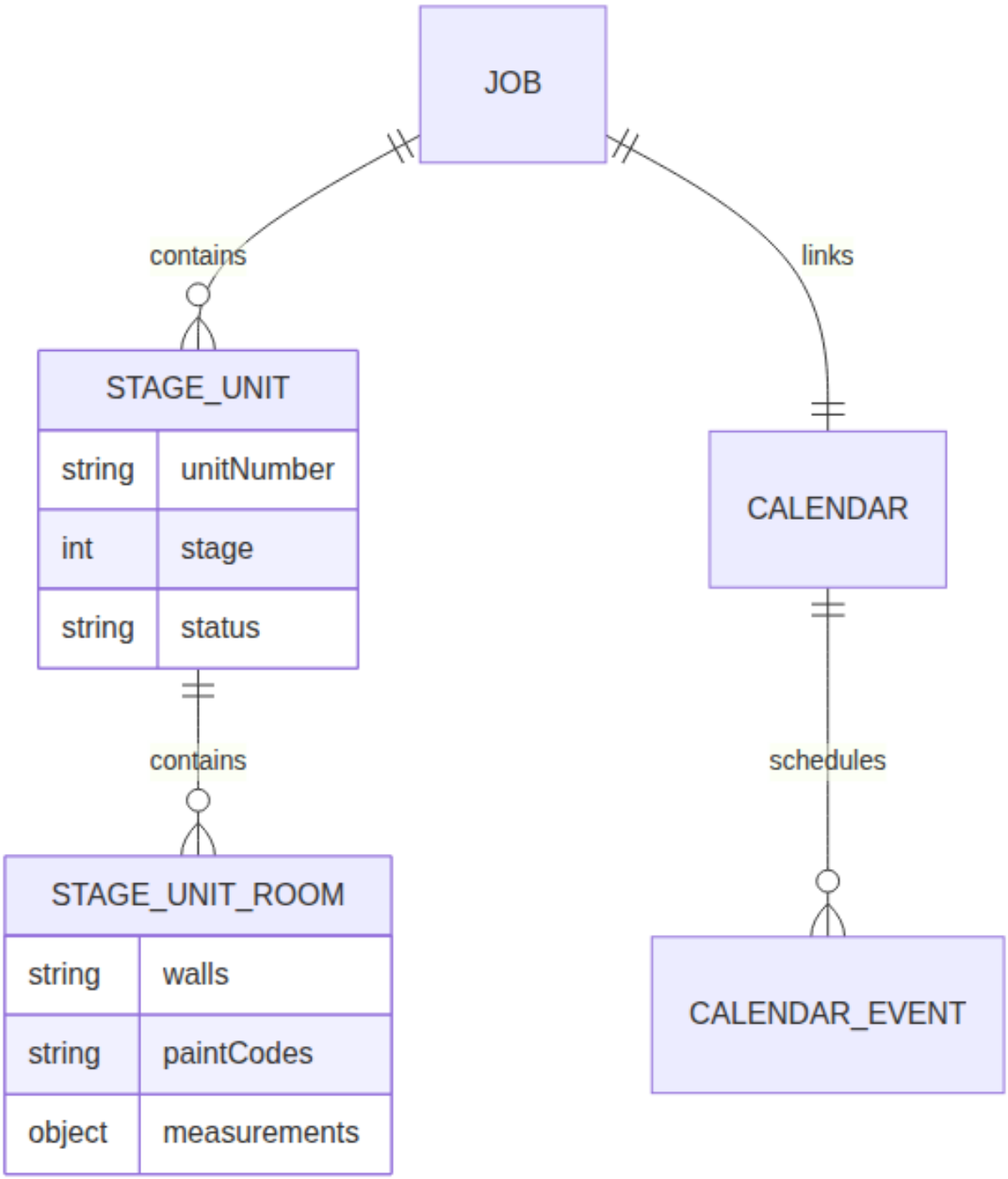



Figure 5: Architecture diagram 5

Middleware: `RequireManagementAPIAccess`, `RequireCalendarAPIAccess`.

**P-005 gap:** authority is role-based, not yet **purpose-scoped** (e.g. scheduling agent may propose but not override safety certification). **P-006 satisfied by absence:** no unbounded agents.

---

# 4. Mapping theory to code

## 4.1 Reference architecture layers vs. repository reality

The manuscript's six layers (Chapter 9) against BCP today:

| Layer | Manuscript intent | BCP today |
|---|---|---|
| **1 — Interfaces** | Capture work with minimal burden | Rich workflow UI; heavy `app.ts` |
| **2 — Observation** | Canonical events, provenance | Direct Mongo writes; calendar store |
| **3 — Org model** | Plural projections, work-object registry | Strong job/unit/room model; weak cross-module registry |
| **4 — Discovery** | Pattern detection, hypotheses | Manual—superintendent's brain |
| **5 — Capability registry** | Versioned definitions, tests, owners | Code in `scheduling/`; no registry |
| **6 — Agent runtime** | Bounded execution | Humans + deterministic services |
| **Governance** | Identity, consent, audit | Sessions, RBAC, CSP; no audit event stream |

## 4.2 Development roadmap alignment (Stages 0–6)

From Capability Architecture Chapter 10, mapped to BCP:

| Stage | Manuscript milestone | BCP status |
|---|---|---|
| **0** | Stable core; no agents | **Achieved** — PlumbingTrack in production |
| **1** | Observation layer | **In progress** — calendar history, checklists; no canonical event envelope |
| **2** | Process understanding | **Planned** — workspace timeline exists; no variant mining |
| **3** | Deterministic capabilities | **Partial** — scheduling engine; no registry |

| Stage | Manuscript milestone | BCP status |
|---|---|---|
| **4** | Assisted discovery | **Future** — model gateway not present |
| **5** | Bounded agents | **Future** |
| **6** | Capability generation | **Research** — explicitly warned against in manuscript |

### 4.3 Gap analysis

**High-signal gaps (honest):**

1. **No canonical event layer** — writes go to Mongo; history is reconstructible with effort, not guaranteed.
2. **Duplicate model boundaries** — `internal/models` vs `internal/plumbingtrack/models` for stage units.
3. **HTML still mostly Go strings** — templ migration started; CI guards exist.
4. **No capability interchange format** — scheduling knowledge locked in Go.
5. **No formal outcome linkage** — did rescheduling improve project delivery? Unknown systematically.
6. **Client job mapping weak** — org-scoped calendar RBAC without persisted client-job graph.

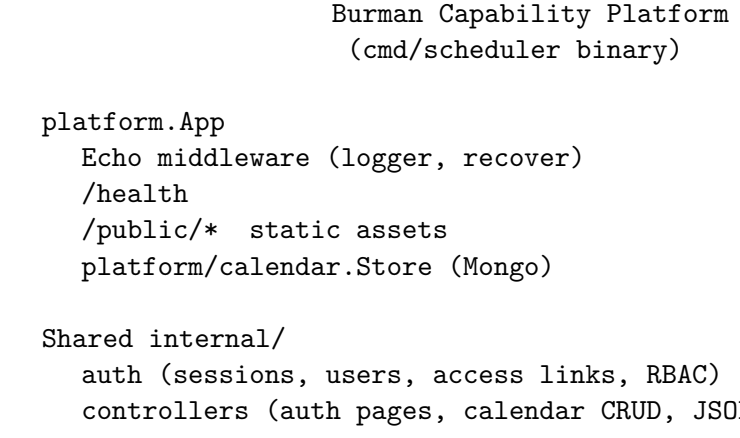
**What is already right:**

1. **`platform.Module` seam** — correct macro-architecture for B2B multi-trade expansion.
2. **Deterministic scheduling** — encodes trade knowledge testably (~37 test files).
3. **Embeds as B2B surface** — white-label calendar distribution without tenant login.
4. **Authority middleware** — foundation for P-005 expansion.
5. **Modularity doc + extraction sequence** — explicit theory-to-repo governance.

---

## 5. Architecture diagrams

### 5.1 As-is: composed server



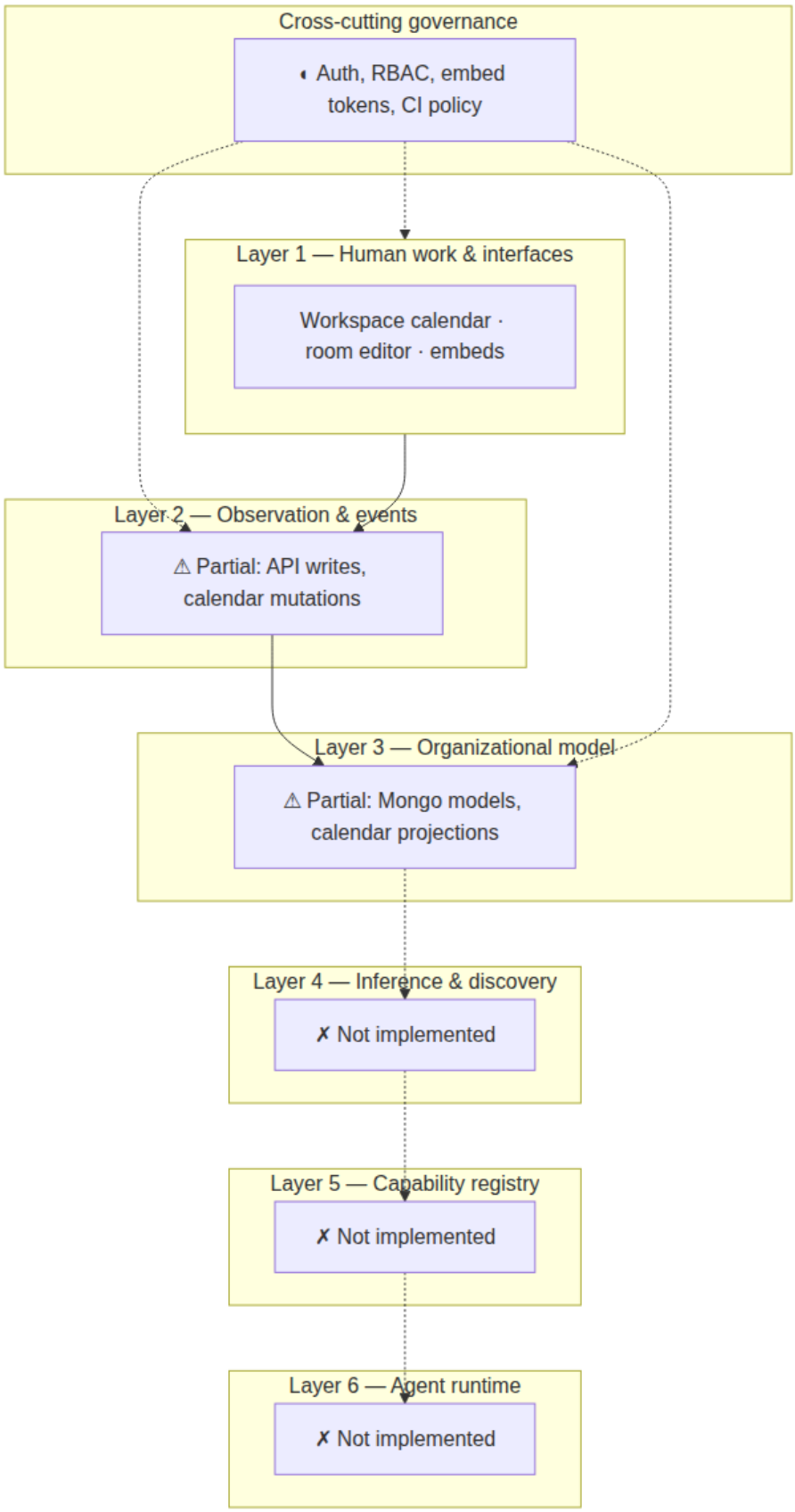



Figure 6: Architecture diagram 6

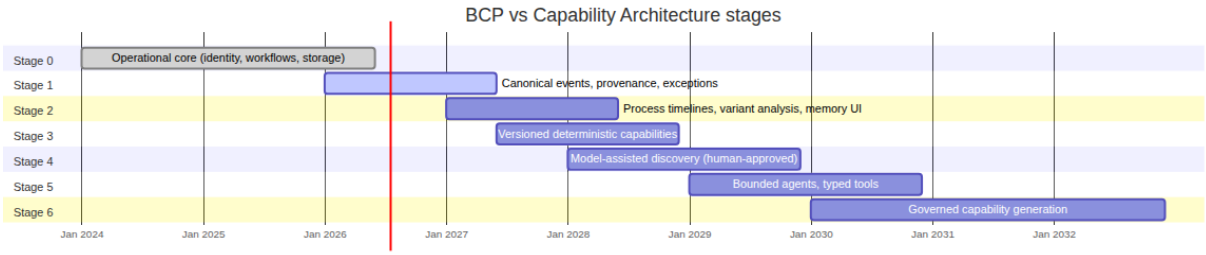



Figure 7: Architecture diagram 7

```
    models (jobs, calendars, shared types)
    config, session, repositories

  plumbingtrack.Module  ("plumbing-track")
    pages/     server-rendered workflow + embeds
    controllers/  REST under /api/v1
    services/  rooms, scheduling persistence
    scheduling/  deterministic stage engine
    stagechecklist/, paintcode/, calendarjob/
    models/    suite, stage unit, room collections
```

```
    MongoDB
  jobs, units,
  calendars,
  sessions, ...
```

## 5.2 Bounded contexts (target)

Manuscript Chapter 10 bounded contexts as **target package topology**:

**BCP mapping path:**

| Bounded context | Current home | Target home |
|---|---|---|
| Identity & Authority | `internal/auth` | `platform/auth` (shared) |
| Organization Registry | `internal/models` (jobs) | `platform/registry` |
| Observation | *scattered* | `platform/observation` |
| Workflow | `plumbingtrack/scheduling` | module-local + platform contracts |
| Memory | Mongo collections | platform memory interfaces |
| Capability Registry | *none* | `platform/capabilities` |
| Agent Runtime | *none* | `platform/agents` (late) |
| Evaluation | tests + manual | `platform/evaluation` |

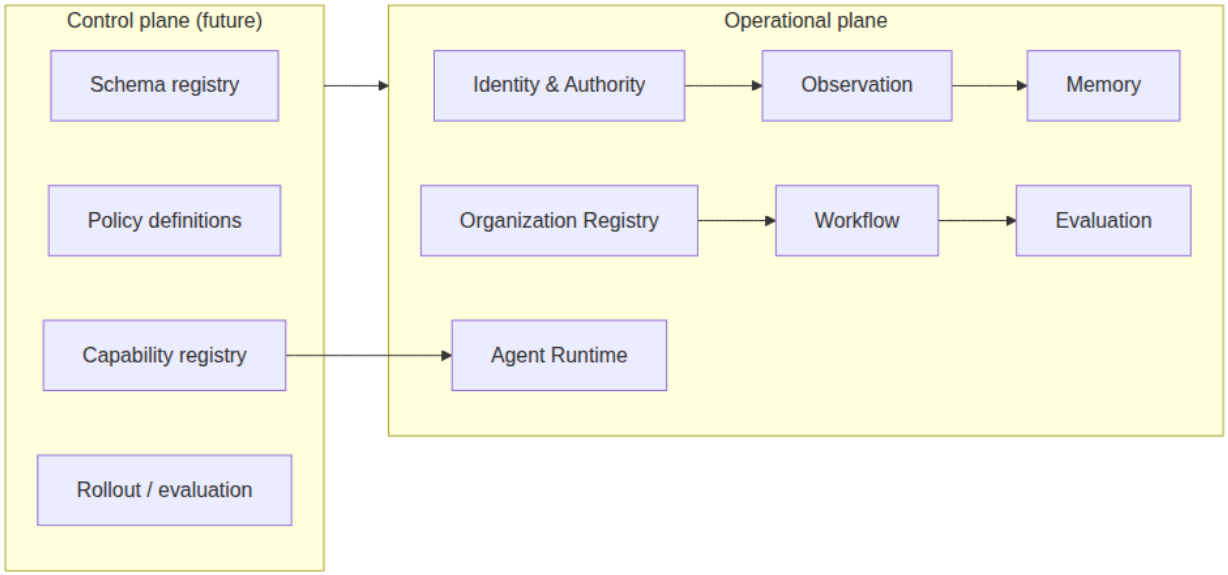



Figure 8: Architecture diagram 8

### 5.3 Capability loop (target)

The closed loop Capability Architecture exists to enable:

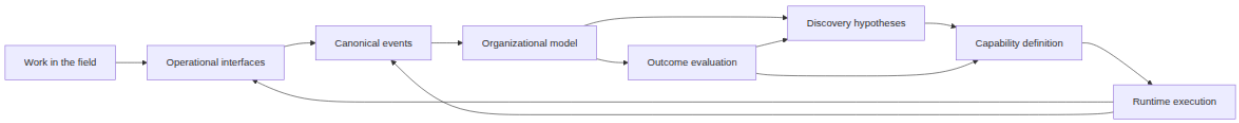



Figure 9: Architecture diagram 9

**PlumbingTrack today** implements the left spine (work → interfaces → model) and a **shortcut** from model → runtime (scheduling) without discovery or formal evaluation. That is appropriate for Stage 0–1.

### 5.4 Multi-tenant B2B topology

Future BCP deployments for multiple contractors and trades:

**Multi-tenancy considerations:**

- **Default:** hard tenant isolation in Mongo (database-per-tenant or collection prefix + enforced `tenant_id`).
- **Shared capability definitions** do not imply shared organizational memory (manuscript warning).
- **Cross-tenant learning** only via explicit licensed artifacts or de-identified aggregates—not silent gradient sharing.

---

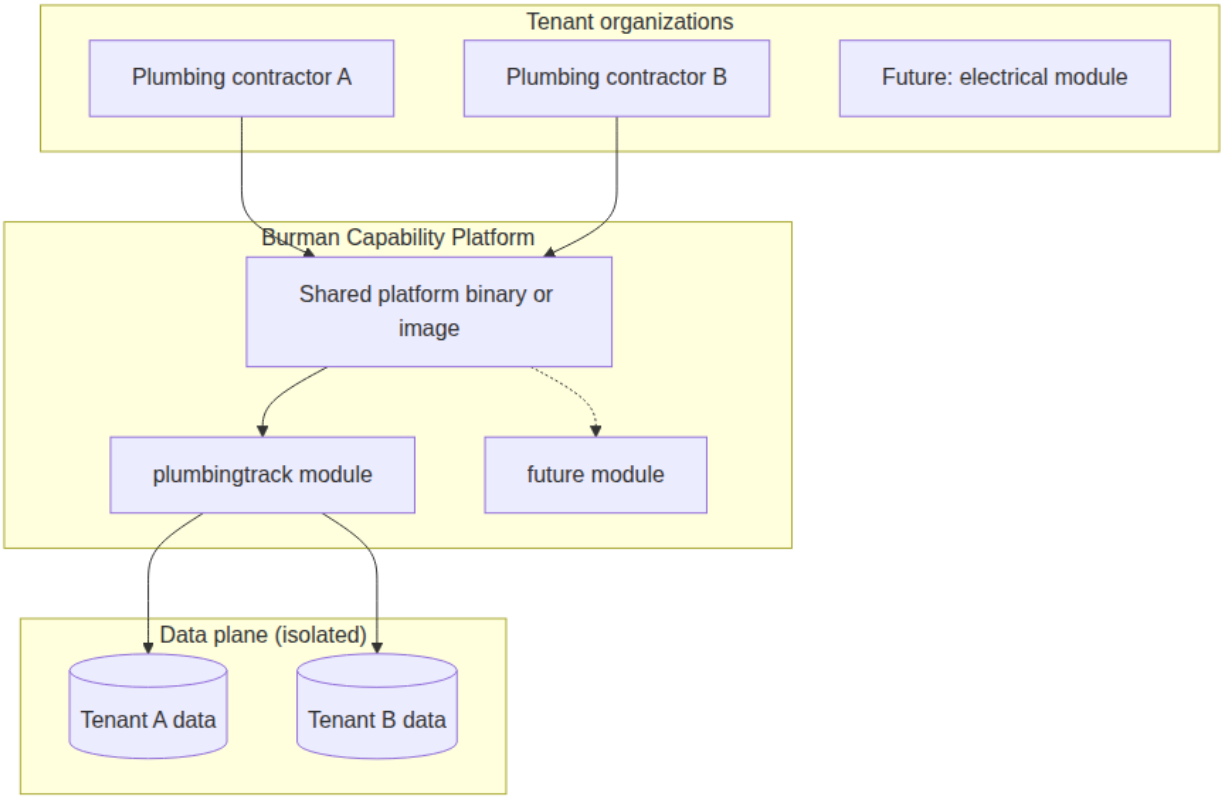



Figure 10: Architecture diagram 10

# 6. Implementation routes

Three viable routes from current repo to Capability Platform product line. They are not mutually exclusive—they differ in **emphasis**.

## 6.1 Route A — Harden the seam, ship PlumbingTrack

**Thesis:** Maximize production value and extraction readiness before infrastructure ambition.

| Work item | Rationale |
|---|---|
| Merge `stage_unit` models/repositories | Eliminate dual boundary confusion |
| templ-first HTML | P-001/P-010: structure before cleverness |
| `platform/auth`, `platform/tenant` contracts | B2B entitlement foundation |
| Docker + `.env.example` + integration tests | Operable product, not just code |
| Extract `plumbing-track-server` repo | Proves `platform.Module` thesis |

**Best when:** revenue and field reliability dominate; team is small.

**Risk:** observation layer delayed; technical debt of direct Mongo writes compounds.

## 6.2 Route B — Observation layer first

**Thesis:** Stage 1 unlocks everything else—without it, agents and discovery are performative.

**Minimal event envelope (from manuscript):**

```
Event {
  id, type, subject_id, actor_id,
  occurred_at, recorded_at, source,
  schema_version, tenant_id, payload, provenance
}
```

**First insertion points in BCP:**

- Stage unit status changes
- Calendar event create/move/delete (cascade)
- Room measurement uploads
- Checklist completion

**Implementation pattern IP-001:** *Plan broadly, commit narrowly*—rich UI context for proposals; narrow domain commands that emit events after policy checks.

**Best when:** the research program matters as much as the product; you believe H-004 (observation reveals unarticulated requirements).

**Risk:** slower feature velocity; team must resist over-engineering projections before use cases exist.

## 6.3 Route C — Extract and multiply modules

**Thesis:** BCP wins as a **platform business**, not a single vertical app.

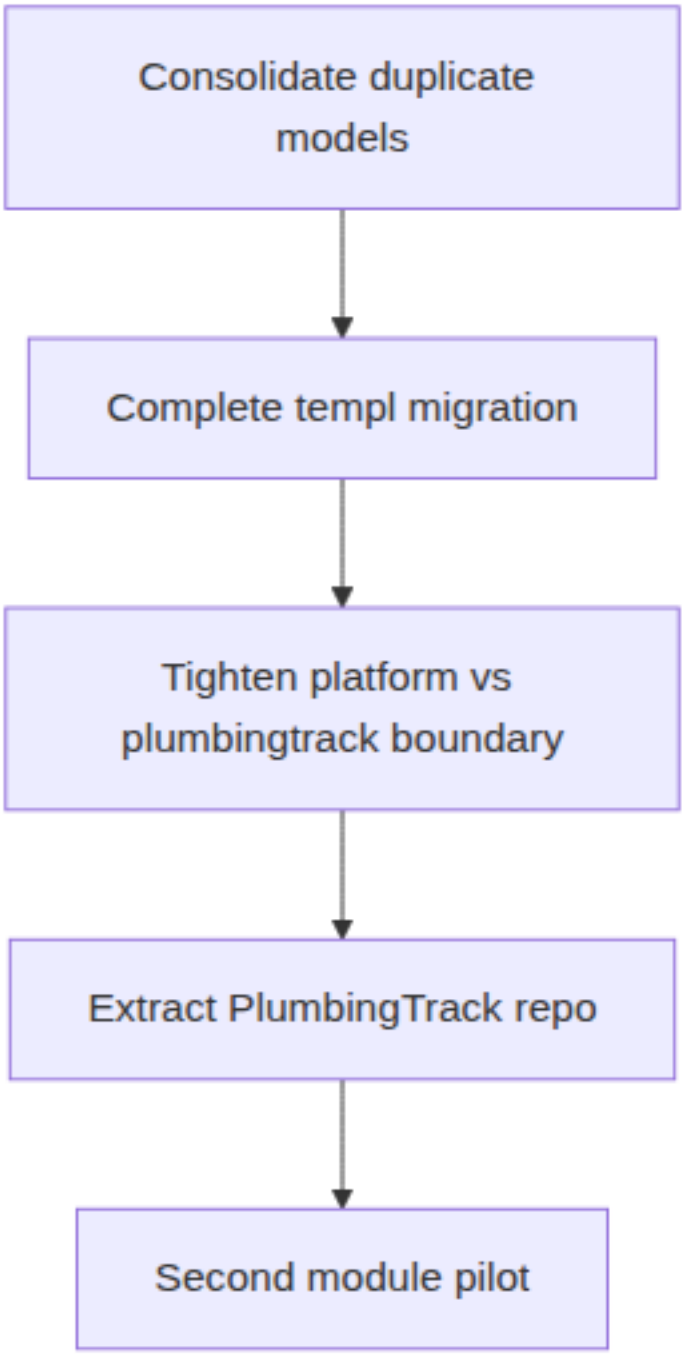



Figure 11: Architecture diagram 11

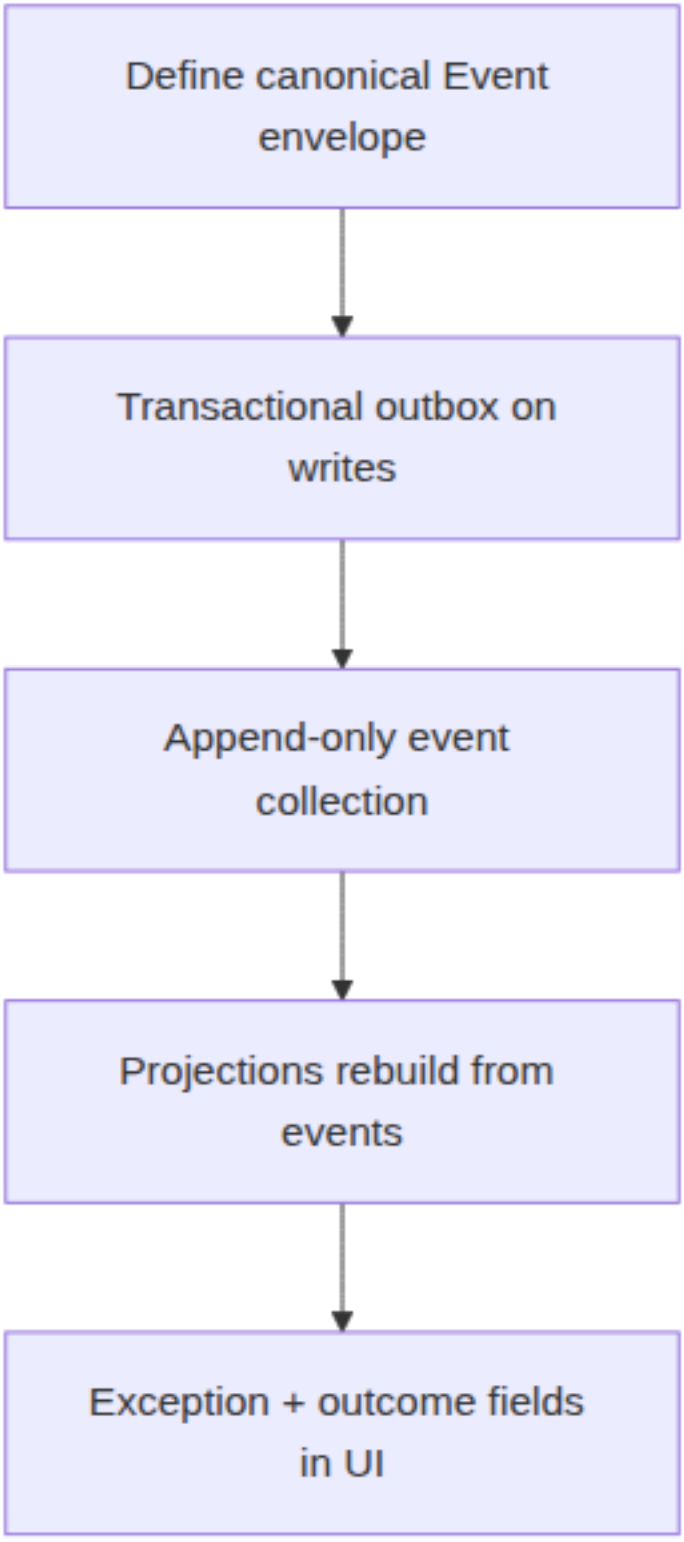



Figure 12: Architecture diagram 12

```
bman-platform          → shared library + platform.App
plumbing-track-server  → first vertical binary
{trade}-track-server   → compile-time siblings
capability-registry    → later: shared definitions service
```

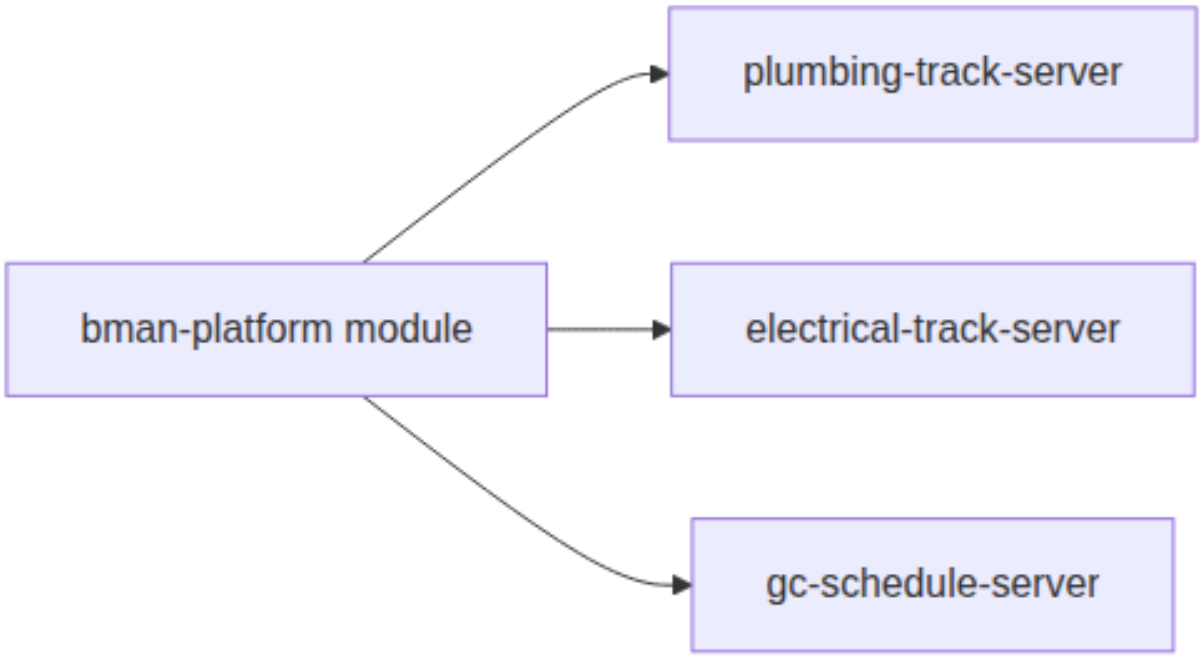



Figure 13: Architecture diagram 13

**Best when:** multiple trades/customers are visible within 18 months; platform licensing is the commercial model.

**Risk:** premature abstraction without two real modules; shared contracts guessed instead of earned.

### 6.4 Route comparison matrix

| Criterion | Route A (seam) | Route B (observation) | Route C (multiply) |
|---|---|---|---|
| Time to revenue | | | |
| Architectural alignment | | | |
| Research value | | | |
| Team size | 3 | | |
| Future agent readiness | | | |
| Commercial platform story | | | |

**Recommended sequencing (pragmatic synthesis):**

```
Now → 6mo:   Route A (harden seam, templ, dedupe models)
6mo → 18mo:  Route B (event envelope on calendar + stage transitions)
18mo → 36mo: Route C (extract vertical, add module #2)
36mo+:       Discovery + bounded agents only if evaluation exists
```

---

# 7. Design considerations

## 7.1 MongoDB today, plural models tomorrow

**P-008** requires event, graph, document, and analytical views—not one database pretending to be the organization.

| View | MongoDB today | Future option |
|---|---|---|
| Transactional state | Collections | PostgreSQL (manuscript reference) |
| Event history | Ad hoc | Append-only `events` collection or table |
| Graph | Implicit in references | Adjacency tables or graph projection |
| Documents | Room photos, notes | Object storage + metadata |
| Analytics | Manual | Read replicas, OLAP projection |

**Migration philosophy:** projections are rebuildable from authoritative events. Mongo can remain the operational store for years if event capture is added—**conceptual boundary matters more than premature Postgres**.

## 7.2 Events: what exists vs. what the theory requires

| Property | BCP today | Required |
|---|---|---|
| Immutability | Updates in place | Append; supersede with new event |
| Provenance | Partial | Actor, source system, authority context |
| Temporal fields | `time.Time` on events | occurred vs recorded vs processed |
| Correlation | Limited | Causal chains for cascade reschedule |
| Schema version | Informal | Explicit per event type |

**Counterexample to avoid:** the Universal JSON Document—one blob with implicit semantics. Events should be **typed, versioned, and governed**.

## 7.3 HTML, templ, and operational interfaces

The **HTML rule** (`docs/modularity.md`): `.templ` is source of truth; no HTML strings in controllers.

Why it matters for Capability Architecture:

- Operational interfaces are **Layer 1 sensors**. Sloppy UI generation creates unmaintainable observation surfaces.
- templ enforces compile-time structure—aligned with **machine-enforceable authority** (forms imply permissions).

CI already runs:

1. `check-templ-generated.sh`
2. `check-no-inline-html.sh`
3. `go test ./...`

The migration is not aesthetic—it is **infrastructure for capability evolution**.

### 7.4 Agents: when, not if—and how bounded

**H-003:** agents are runtime realizations of capability definitions—not chat interfaces.

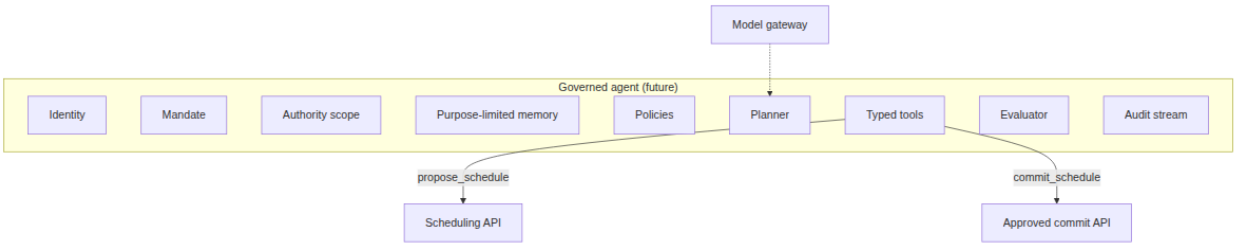



Figure 14: Architecture diagram 14

**BCP agent candidates (Stage 5, not before):**

| Agent | Mandate | Tools | Authority ceiling |
|---|---|---|---|
| Schedule proposer | Suggest cascade fixes | `propose_schedule` | Read-only on calendars |
| Exception triage | Cluster repeated frictions | `search_events` | No writes |
| Checklist assistant | Draft sub-task dates | `propose_checklist` | Human commit required |

**Failure modes to engineer against:** authority leakage, context contamination, goal drift, invisible delegation (Chapter 8).

### 7.5 B2B packaging: tenants, entitlements, white-label

Commercial BCP needs:

| Concern | Mechanism |
|---|---|
| Tenant isolation | `tenant_id` on all work objects + DB isolation |
| Module entitlements | Compile-time module + runtime feature flags |

| Concern | Mechanism |
|---|---|
| White-label embeds | `EMBED_FRAME_ANCESTORS`, custom CSS hooks |
| Client visibility | Persisted client job mapping (improvement #7 in README) |
| Audit | Event stream for B2B compliance narratives |

---

# 8. Future prospects

## 8.1 Near term (6–12 months)

**Product:** PlumbingTrack as the flagship B2B vertical—reliable scheduling, room intelligence, embed calendars for building stakeholders.

**Engineering:**

- templ migration for `pages/` and auth shells
- Model consolidation (single stage unit boundary)
- Integration tests with MongoDB testcontainers
- Docker Compose dev stack
- Extract PlumbingTrack into sibling repo importing `bman-platform`

**Capability Architecture alignment:** complete Stage 0; begin Stage 1 with event envelope on calendar mutations.

## 8.2 Medium term (12–24 months)

**Product:** Burman Capability Platform as a **named platform**—second trade module in pilot; capability registry for scheduling rules as versioned artifacts.

**Engineering:**

- `platform/observation` package; rebuild calendar projections from events
- Process timeline UI—compare expected workflow vs actual event history
- Outcome fields on jobs (on-time stage completion, rework counts)
- Model gateway for **assisted** exception summarization—human approval required (Stage 4 entry)

**Commercial:** per-tenant SaaS; licensed embeds; optional on-prem for large contractors.

## 8.3 Long term: capability systems, not app catalogs

The manuscript's conclusion is the north star:

> *Operational software becomes the sensory and actuating surface of the organization. Event histories become evidence. Organizational models become provisional explanations. Capability definitions become versioned units of change. Agents become bounded runtime realizations.*

**BCP long-term prospects if foundations hold:**

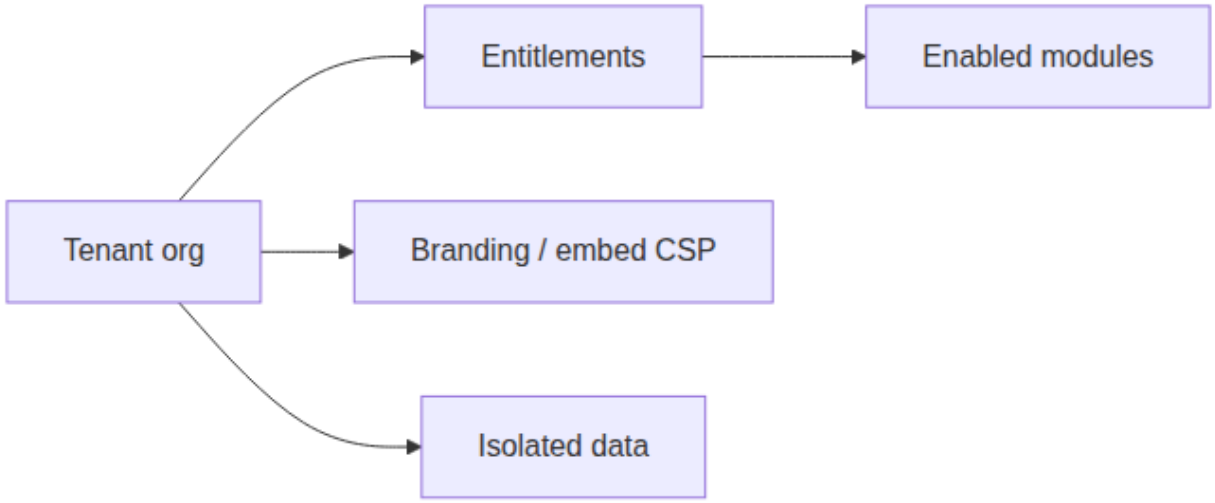



Figure 15: Architecture diagram 15

1. **Capability marketplace** (H-007) — licensed scheduling/regulatory capabilities across contractors, with explicit adaptation obligations.
2. **Cross-org learning without surveillance** — de-identified friction patterns; no silent training on tenant data.
3. **Composable trade operations** — plumbing + electrical + GC scheduling share work-object registry and calendar primitives.
4. **Governed code generation (Stage 6)** — only where registry, simulation, and evaluation are mature—not demo-ware.

**If foundations fail:** BCP still wins as excellent vertical operations software. The theory is falsifiable; the plumbing scheduler still saves superintendents hours. P-001 demands honesty about which outcome occurred.

---

## 9. Research program and falsification

Capability Architecture explicitly invites criticism. BCP is the instrument.

| Hypothesis | Test on BCP |
|---|---|
| **H-001** Dynamic knowledge graphs represent organizations | Model jobs/units/events as temporal graph; query friction patterns |
| **H-002** Capability beats feature as reusable unit | Version scheduling as capability definition; measure transfer to second building typology |
| **H-004** Observation reveals tacit requirements | Instrument exceptions; compare discovered rules to interview baselines |
| **H-005** Friction appears as repeated exception handling | Mine reschedule + dedupe logs |
| **H-008** Outcome-linked histories reduce learning cost | A/B on projects with/without structured outcome capture |

**Success criteria (from manuscript):** reduce effort to understand current work; preserve disappearing knowledge; identify recurring friction; support safer automation; measure better outcomes—not model engagement.

**Failure modes to document publicly:** surveillance creep, proxy metric optimization, capability transfer that imposes source-org assumptions, agent deployments without evaluation.

---

## 10. Conclusion

Burman Capability Platform is where **philosophy meets plumbing**—literally. The Capability Architecture manuscript asks how software can participate in organizational learning without mistaking the map for the territory. BCP answers with a concrete, shippable stack: Go, Echo, MongoDB, compile-time modules, deterministic scheduling, and a modularity seam that already matches the theory's "platform as first implementation."

The current system is **not** an adaptive capability system yet. It is **Stage 0 software with Stage 1 ambitions**—and that is the correct honesty. The scheduling engine, embed surface, RBAC middleware, and `platform.Module` boundary are seeds. The canonical event layer, capability registry, and bounded agents are roots not yet planted.

The juicy opportunity is not slapping a chat box on a calendar. It is building B2B infrastructure where every interaction produces **governed evidence**, every trade module shares **identity and authority primitives**, and every automation must eventually answer: **what capability did we improve, for whom, measured how?**

That is a harder product story than "AI-powered scheduling." It is also the one worth building.

---

## 11. Appendix: key sources

| Source | Role |
|---|---|
| `docs/The_Capability_Architecture_Volume_I_Foundations_v0.3.pdf` | Normative theory (principles, definitions, roadmap) |
| `docs/modularity.md` | Platform extraction sequence and HTML rule |
| `README.md` | Current technical surface and opportunities |
| `platform/app.go` | `Module` interface and shared server |
| `internal/plumbingtrack/module.go` | First capability module registration |
| `internal/plumbingtrack/scheduling/` | Deterministic proto-capability |
| `graphify-out/graph.json` | Cross-linked knowledge graph (theory code) |

---

*Copyright © 2026 Sean Burman. Working thesis for platform development and research discussion.*